COPYRIGHT SOURCE CODE SAMPLES

C.A.R.E.N. Intellectual Property Documentation

C.A.R.E.N. Source Code Samples for Copyright Registration

Representative Code Samples Demonstrating Unique Copyrightable Expression

FIRST 25 PAGES - CORE ALGORITHM IMPLEMENTATIONS

Page 1-3: GPS-Legal Rights Correlation Engine (server/aiService.ts)

```typescript
import Anthropic from '@anthropic-ai/sdk';
import { db } from './db';
import { legalRights, incidents, type LegalRights, type Incident } from
'@shared/schema';
import { eq, and } from 'drizzle-orm';

const anthropic = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY,
});

interface AIQuestionRequest {
  question: string;
  context?: {
    userState?: string;
    userLocation?: string;
    recentIncidents?: Incident[];
    availableRights?: LegalRights[];
  };
}

interface AIResponse {
  answer: string;
  confidence: number;
  suggestions?: string[];
  relatedRights?: string[];
  disclaimer: string;
}

export class AILegalAssistant {
  static async answerQuestion(request: AIQuestionRequest): Promise {
    const { question, context } = request;

    // Build comprehensive context for AI analysis
    let contextString = "";

    if (context?.userState) {
      contextString += User is located in: ${context.userState}\n;
    }

    if (context?.userLocation) {
      contextString += Current location: ${context.userLocation}\n;
    }

    if (context?.availableRights && context.availableRights.length > 0) {
      contextString += "\nRelevant Legal Rights:\n";
```

```
      context.availableRights.forEach(right => {
        contextString += - ${right.title}: ${right.content}\n;
      });
    }

    if (context?.recentIncidents && context.recentIncidents.length > 0) {
      contextString += "\nRecent User Incidents:\n";
      context.recentIncidents.forEach(incident => {
        contextString += - ${incident.type} on ${incident.createdAt}\n;
      });
    }

     const prompt = `You are C.A.R.E.N., an expert legal assistant specializing
in constitutional rights and police encounters.

Context:
${contextString}

User Question: ${question}

Provide a helpful, accurate response that:

• Directly answers their question

• Cites relevant legal rights when applicable

• Provides practical guidance for police encounters

• Includes appropriate legal disclaimers

• Suggests related rights or actions they should know

Be concise but thorough. Always prioritize user safety.`;

    try {
      const response = await anthropic.messages.create({
        model: "claude-sonnet-4-20250514",
        max_tokens: 1024,
        messages: [{ role: 'user', content: prompt }],
      });

      const answer = response.content[0].text;

      // Extract suggestions and related rights from the response
      const suggestions = this.extractSuggestions(answer);
                  const   relatedRights   =   this.extractRelatedRights(answer,
context?.availableRights || []);
```

```
      return {
        answer,
        confidence: 0.85, // Base confidence for AI responses
        suggestions,
        relatedRights,
        disclaimer: "This information is for educational purposes only and does not constitute legal advice. Consult with a qualified attorney for specific legal guidance."
      };
    } catch (error) {
      console.error('AI Legal Assistant error:', error);
      return {
        answer: "I'm unable to process your question right now. Please contact an attorney for immediate legal assistance.",
        confidence: 0,
        suggestions: ["Contact an attorney", "Review your local legal rights", "Document any incidents"],
        relatedRights: [],
        disclaimer: "This system is temporarily unavailable. Seek professional legal counsel for urgent matters."
      };
    }
  }

  private static extractSuggestions(response: string): string[] {
    // Extract actionable suggestions from AI response
    const suggestionPatterns = [
      /You should (.*?)(?:\.|$)/gi,
      /Consider (.*?)(?:\.|$)/gi,
      /Remember to (.*?)(?:\.|$)/gi,
      /Make sure to (.*?)(?:\.|$)/gi
    ];

    const suggestions: string[] = [];

    suggestionPatterns.forEach(pattern => {
      const matches = response.match(pattern);
      if (matches) {
        matches.forEach(match => {
          const suggestion = match.replace(pattern, '$1').trim();
          if (suggestion.length > 10 && suggestion.length < 100) {
            suggestions.push(suggestion);
          }
        });
      }
    });
```

```
    return suggestions.slice(0, 5); // Limit to 5 suggestions
  }

    private  static  extractRelatedRights(response:  string,  availableRights:
LegalRights[]): string[] {
    const relatedRights: string[] = [];

    availableRights.forEach(right => {
      const keywords = right.title.toLowerCase().split(' ');
      const responseText = response.toLowerCase();

      let matchCount = 0;
      keywords.forEach(keyword => {
        if (keyword.length > 3 && responseText.includes(keyword)) {
          matchCount++;
        }
      });

      // If multiple keywords match, consider it related
      if (matchCount >= 2) {
        relatedRights.push(right.title);
      }
    });

    return relatedRights.slice(0, 3); // Limit to 3 related rights
  }
}
```

Page 4-6: Voice Command Recognition System (client/src/hooks/useVoiceCommands.ts)

```typescript
import { useState, useEffect, useCallback, useRef } from 'react';
import { useToast } from './use-toast';

interface VoiceCommand {
  pattern: string;
  action: string;
  description: string;
  category: 'navigation' | 'emergency' | 'rights' | 'recording';
}

interface VoiceCommandResult {
  command: string;
  confidence: number;
  action: string;
  timestamp: Date;
}

const VOICE_COMMANDS: VoiceCommand[] = [
  // Navigation Commands
   { pattern: 'go to dashboard', action: 'navigate:/dashboard', description: 'Navigate to main dashboard', category: 'navigation' },
   { pattern: 'show my rights', action: 'navigate:/legal-rights', description: 'Display legal rights page', category: 'navigation' },
     { pattern: 'emergency mode', action: 'navigate:/emergency-pullover', description: 'Activate emergency mode', category: 'emergency' },

  // Emergency Commands
   { pattern: 'start recording', action: 'start-recording', description: 'Begin incident recording', category: 'recording' },
   { pattern: 'stop recording', action: 'stop-recording', description: 'End incident recording', category: 'recording' },
    { pattern: 'call attorney', action: 'emergency-attorney', description: 'Contact emergency attorney', category: 'emergency' },
   { pattern: 'emergency alert', action: 'send-emergency-alert', description: 'Send alert to emergency contacts', category: 'emergency' },

  // Constitutional Rights Commands
   { pattern: 'I invoke my fourth amendment rights', action: 'invoke-fourth-amendment', description: 'Invoke 4th Amendment protection', category: 'rights' },
   { pattern: 'I invoke my fifth amendment rights', action: 'invoke-fifth-amendment', description: 'Invoke 5th Amendment protection', category: 'rights' },
   { pattern: 'I do not consent to searches', action: 'refuse-search-consent', description: 'Refuse consent to search', category: 'rights' },
```

```
  { pattern: 'I want to remain silent', action: 'invoke-silence', description:
'Invoke right to remain silent', category: 'rights' },
  { pattern: 'am I free to leave', action: 'ask-detention-status', description:
'Inquire about detention status', category: 'rights' },

  // Advanced Emergency Patterns
  { pattern: 'help me', action: 'emergency-assistance', description: 'Request
immediate assistance', category: 'emergency' },
  { pattern: 'I need help', action: 'emergency-assistance', description:
'Request immediate assistance', category: 'emergency' },
  { pattern: 'police stop', action: 'activate-police-stop-mode', description:
'Activate police stop protocol', category: 'emergency' },
  { pattern: 'traffic stop', action: 'activate-traffic-stop-mode', description:
'Activate traffic stop protocol', category: 'emergency' },
];

export const useVoiceCommands = () => {
  const [isListening, setIsListening] = useState(false);
  const [isSupported, setIsSupported] = useState(false);
  const [lastCommand, setLastCommand] = useState(null);
  const [commandHistory, setCommandHistory] = useState([]);
  const { toast } = useToast();

  const recognitionRef = useRef(null);
  const isProcessingRef = useRef(false);

  // Initialize speech recognition
  useEffect(() => {
    if (typeof window !== 'undefined') {
                  const SpeechRecognition = window.SpeechRecognition ||
window.webkitSpeechRecognition;

      if (SpeechRecognition) {
        setIsSupported(true);
        const recognition = new SpeechRecognition();

        recognition.continuous = true;
        recognition.interimResults = false;
        recognition.lang = 'en-US';
        recognition.maxAlternatives = 3;

        recognition.onstart = () => {
          console.log('Voice recognition started');
          setIsListening(true);
        };
```

```
        recognition.onend = () => {
          console.log('Voice recognition ended');
          setIsListening(false);
          isProcessingRef.current = false;
        };

        recognition.onerror = (event) => {
          console.error('Voice recognition error:', event.error);
          setIsListening(false);
          isProcessingRef.current = false;

          if (event.error === 'not-allowed') {
            toast({
              title: "Microphone Access Denied",
               description: "Please enable microphone access for voice commands
to work.",
              variant: "destructive",
            });
          }
        };

        recognition.onresult = (event) => {
          if (isProcessingRef.current) return;
          isProcessingRef.current = true;

          const results = Array.from(event.results);
          const latestResult = results[results.length - 1];

          if (latestResult.isFinal) {
            const transcript = latestResult[0].transcript.toLowerCase().trim();
            const confidence = latestResult[0].confidence;

              console.log('Voice command detected:', transcript, 'Confidence:',
confidence);

            // Process the voice command
            processVoiceCommand(transcript, confidence);
          }
        };

        recognitionRef.current = recognition;
      } else {
        setIsSupported(false);
        console.warn('Speech recognition not supported in this browser');
      }
```

```
    }
  }, []);

    const processVoiceCommand = useCallback((transcript: string, confidence: number) => {
    let bestMatch: VoiceCommand | null = null;
    let highestScore = 0;

    // Find the best matching command using fuzzy matching
    VOICE_COMMANDS.forEach(command => {
      const score = calculateCommandScore(transcript, command.pattern);
        if (score > highestScore && score > 0.6) { // Minimum confidence threshold
        highestScore = score;
        bestMatch = command;
      }
    });

    if (bestMatch) {
      const commandResult: VoiceCommandResult = {
        command: bestMatch.pattern,
        confidence: confidence * highestScore,
        action: bestMatch.action,
        timestamp: new Date()
      };

      setLastCommand(commandResult);
        setCommandHistory(prev => [...prev.slice(-9), commandResult]); // Keep last 10 commands

      // Execute the command
      executeVoiceCommand(bestMatch);

      // Provide user feedback
      toast({
        title: "Voice Command Recognized",
        description: Executing: ${bestMatch.description},
        duration: 2000,
      });

      console.log('Executing voice command:', bestMatch);
    } else {
      console.log('No matching voice command found for:', transcript);

      // Check if it might be a constitutional rights phrase
      if (isConstitutionalRightsPhrase(transcript)) {
```

```
        handleConstitutionalRightsInvocation(transcript);
      }
    }

    isProcessingRef.current = false;
  }, [toast]);

  const calculateCommandScore = (transcript: string, pattern: string): number
=> {
    // Implement fuzzy string matching algorithm
    const transcriptWords = transcript.split(' ');
    const patternWords = pattern.split(' ');

    let matchedWords = 0;
    let totalWords = patternWords.length;

    patternWords.forEach(patternWord => {
      const matches = transcriptWords.some(transcriptWord => {
        // Exact match
        if (transcriptWord === patternWord) return true;

        // Partial match (for longer words)
                                    if    (patternWord.length    >    4    &&
transcriptWord.includes(patternWord.slice(0, -1))) return true;

        // Sound-alike matching for common substitutions
        const soundAlikes = {
          'fourth': ['4th', 'forth'],
          'fifth': ['5th'],
          'amendment': ['amendment'],
          'search': ['searches'],
          'consent': ['consents']
        };

        if (soundAlikes[patternWord]?.includes(transcriptWord)) return true;

        return false;
      });

      if (matches) matchedWords++;
    });

    return matchedWords / totalWords;
  };
```

```
  const isConstitutionalRightsPhrase = (transcript: string): boolean => {
    const rightsKeywords = [
      'amendment', 'constitutional', 'rights', 'remain silent',
      'do not consent', 'refuse', 'invoke', 'fourth', 'fifth'
    ];

    return rightsKeywords.some(keyword => transcript.includes(keyword));
  };

  const handleConstitutionalRightsInvocation = (transcript: string) => {
    // Log constitutional rights invocation for legal documentation
    const rightsInvocation = {
      transcript,
      timestamp: new Date(),
      type: 'constitutional_rights_invocation'
    };

    // Store in incident documentation
                                  localStorage.setItem('last_rights_invocation',
JSON.stringify(rightsInvocation));

    toast({
      title: "Constitutional Rights Invoked",
      description: "Your rights invocation has been documented.",
      duration: 3000,
    });

    console.log('Constitutional rights invoked:', rightsInvocation);
  };

  const executeVoiceCommand = (command: VoiceCommand) => {
    const { action } = command;

    if (action.startsWith('navigate:')) {
      const path = action.replace('navigate:', '');
      window.location.href = path;
    } else {
      // Handle other command types
      switch (action) {
        case 'start-recording':
          // Trigger recording start
                     window.dispatchEvent(new  CustomEvent('voice-command-start-
recording'));
          break;

        case 'stop-recording':
```

```
          // Trigger recording stop
                      window.dispatchEvent(new  CustomEvent('voice-command-stop-
recording'));
          break;

        case 'emergency-attorney':
          // Navigate to attorney contact
          window.location.href = '/attorney-contact';
          break;

        case 'send-emergency-alert':
          // Trigger emergency alert
                 window.dispatchEvent(new  CustomEvent('voice-command-emergency-
alert'));
          break;

        case 'invoke-fourth-amendment':
        case 'invoke-fifth-amendment':
        case 'refuse-search-consent':
        case 'invoke-silence':
        case 'ask-detention-status':
          // Document rights invocation
          handleConstitutionalRightsInvocation(command.pattern);
          break;

        default:
          console.log('Unknown voice command action:', action);
      }
    }
  };

  const startListening = useCallback(() => {
    if (recognitionRef.current && isSupported && !isListening) {
      try {
        recognitionRef.current.start();
      } catch (error) {
        console.error('Error starting voice recognition:', error);
      }
    }
  }, [isSupported, isListening]);

  const stopListening = useCallback(() => {
    if (recognitionRef.current && isListening) {
      recognitionRef.current.stop();
    }
  }, [isListening]);
```

```
  const toggleListening = useCallback(() => {
    if (isListening) {
      stopListening();
    } else {
      startListening();
    }
  }, [isListening, startListening, stopListening]);

  return {
    isListening,
    isSupported,
    lastCommand,
    commandHistory,
    startListening,
    stopListening,
    toggleListening,
    availableCommands: VOICE_COMMANDS
  };
};
```

Page 7-10: Multi-Device Bluetooth Coordination (server/bluetoothEarpieceService.ts)

```typescript
import { db } from './db';
import { bluetoothDevices, bluetoothSettings, type BluetoothDevice, type
InsertBluetoothDevice } from '@shared/schema';
import { eq, and } from 'drizzle-orm';

export interface BluetoothDevice {
  id: string;
  name: string;
  type: 'earpiece' | 'headphones' | 'speaker' | 'car_audio' | 'unknown';
  connected: boolean;
  batteryLevel?: number;
  signalStrength: number;
  capabilities: {
    audio: boolean;
    microphone: boolean;
    voiceCommands: boolean;
    emergencyButton: boolean;
  };
  lastConnected?: string;
  trustLevel: 'trusted' | 'new' | 'suspicious';
  deviceAddress: string;
}

export interface BluetoothConnectionResult {
  success: boolean;
  deviceId: string;
  deviceName: string;
  connectionTime?: number;
  audioQuality?: 'excellent' | 'good' | 'fair' | 'poor';
  error?: string;
}

export interface AudioStreamConfig {
  sampleRate: number;
  channels: number;
  bitRate: number;
  codec: 'SBC' | 'AAC' | 'aptX' | 'LDAC';
  latency: number;
}

export interface EmergencyProtocol {
  activationPhrase: string;
  actions: string[];
  priority: 'critical' | 'high' | 'medium';
  autoRecord: boolean;
```

```
  contactEmergencyServices: boolean;
  notifyEmergencyContacts: boolean;
}

export class BluetoothEarpieceService {
  /
   * Get all available Bluetooth devices for a user
   */
  static async getUserBluetoothDevices(userId: string): Promise {
    try {
      const devices = await db
        .select()
        .from(bluetoothDevices)
        .where(eq(bluetoothDevices.userId, userId));

      return devices.map(device => ({
        id: device.id,
        name: device.deviceName,
        type: device.deviceType as any,
        connected: device.isConnected,
        batteryLevel: device.batteryLevel || undefined,
        signalStrength: device.signalStrength,
        capabilities: device.capabilities as any,
        lastConnected: device.lastConnected?.toISOString(),
        trustLevel: device.trustLevel as any,
        deviceAddress: device.deviceAddress
      }));
    } catch (error) {
      console.error('Error fetching Bluetooth devices:', error);
      return [];
    }
  }

  /
   * Register a new Bluetooth device
   */
  static async registerBluetoothDevice(
    userId: string,
    deviceInfo: {
      name: string;
      type: string;
      deviceAddress: string;
      capabilities?: any;
    }
  ): Promise {
    const deviceData: InsertBluetoothDevice = {
      id: bt-${Date.now()}-${Math.random().toString(36).substr(2, 9)},
```

```
      userId,
      deviceName: deviceInfo.name,
      deviceType: deviceInfo.type,
      deviceAddress: deviceInfo.deviceAddress,
      isConnected: false,
      signalStrength: 0,
      capabilities: deviceInfo.capabilities || {
        audio: true,
        microphone: true,
        voiceCommands: false,
        emergencyButton: false
      },
      trustLevel: 'new',
      createdAt: new Date(),
      updatedAt: new Date()
    };

    const [device] = await db
      .insert(bluetoothDevices)
      .values(deviceData)
      .returning();

    return {
      id: device.id,
      name: device.deviceName,
      type: device.deviceType as any,
      connected: device.isConnected,
      batteryLevel: device.batteryLevel || undefined,
      signalStrength: device.signalStrength,
      capabilities: device.capabilities as any,
      lastConnected: device.lastConnected?.toISOString(),
      trustLevel: device.trustLevel as any,
      deviceAddress: device.deviceAddress
    };
  }

  /
   * Connect to a Bluetooth device with enhanced retry logic
   */
  static async connectToDevice(userId: string, deviceId: string): Promise {
    const maxAttempts = 3;
    let lastError = '';

    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
      try {
         console.log(Bluetooth connection attempt ${attempt}/${maxAttempts} for
device ${deviceId});
```

```
        const device = await db
          .select()
          .from(bluetoothDevices)
          .where(and(
            eq(bluetoothDevices.id, deviceId),
            eq(bluetoothDevices.userId, userId)
          ))
          .limit(1);

        if (!device.length) {
          return {
            success: false,
            deviceId,
            deviceName: 'Unknown',
            error: 'Device not found'
          };
        }

        const bluetoothDevice = device[0];

        // Simulate connection process with progressive timeout
         const connectionTimeout = 1000 + (attempt * 500); // Increase timeout
with each attempt
        const connectionSuccess = await this.simulateBluetoothConnection(
          bluetoothDevice.deviceAddress,
          connectionTimeout
        );

        if (connectionSuccess) {
          // Update device connection status
          await db
            .update(bluetoothDevices)
            .set({
              isConnected: true,
              lastConnected: new Date(),
                signalStrength: Math.floor(Math.random() * 30) + 70, // 70-100%
signal
              updatedAt: new Date()
            })
            .where(eq(bluetoothDevices.id, deviceId));

                                                    const     audioQuality     =
this.determineAudioQuality(bluetoothDevice.signalStrength);

          console.log(Bluetooth connection successful on attempt ${attempt});

          return {
```

```
            success: true,
            deviceId,
            deviceName: bluetoothDevice.deviceName,
            connectionTime: connectionTimeout,
            audioQuality
          };
        } else {
          lastError = Connection failed on attempt ${attempt};
          console.log(lastError);

          // Wait before retry (except on last attempt)
          if (attempt < maxAttempts) {
            await new Promise(resolve => setTimeout(resolve, 500 * attempt));
          }
        }
      } catch (error) {
        lastError = Error on attempt ${attempt}: ${error.message};
        console.error(lastError);

        if (attempt < maxAttempts) {
          await new Promise(resolve => setTimeout(resolve, 500 * attempt));
        }
      }
    }

    return {
      success: false,
      deviceId,
      deviceName: 'Connection Failed',
      error: Failed after ${maxAttempts} attempts. ${lastError}
    };
  }

  /
   * Enhanced Bluetooth connection simulation with realistic behavior
   */
  private static async simulateBluetoothConnection(
    deviceAddress: string,
    timeout: number = 1000
  ): Promise {
    return new Promise((resolve) => {
      // Simulate realistic connection behavior
         const connectionDelay = Math.random()   timeout   0.8; // Use 80% of
timeout

      setTimeout(() => {
```

```
        // Higher success rate for subsequent attempts
        const successRate = 0.85; // 85% success rate
        const isSuccessful = Math.random() < successRate;

          console.log(Bluetooth simulation result: ${isSuccessful ? 'success' :
'failure'} after ${connectionDelay.toFixed(0)}ms);
        resolve(isSuccessful);
      }, connectionDelay);
    });
  }

  /
   * Determine audio quality based on signal strength
   */
   private static determineAudioQuality(signalStrength: number): 'excellent' |
'good' | 'fair' | 'poor' {
    if (signalStrength >= 80) return 'excellent';
    if (signalStrength >= 60) return 'good';
    if (signalStrength >= 40) return 'fair';
    return 'poor';
  }

  /
   * Disconnect from a Bluetooth device
   */
   static async disconnectFromDevice(userId: string, deviceId: string): Promise
{
    try {
      await db
        .update(bluetoothDevices)
        .set({
          isConnected: false,
          signalStrength: 0,
          updatedAt: new Date()
        })
        .where(and(
          eq(bluetoothDevices.id, deviceId),
          eq(bluetoothDevices.userId, userId)
        ));

      return true;
    } catch (error) {
      console.error('Error disconnecting Bluetooth device:', error);
      return false;
    }
  }
```

```
/
 * Get Bluetooth settings for a user
 */
static async getBluetoothSettings(userId: string) {
  try {
    const settings = await db
      .select()
      .from(bluetoothSettings)
      .where(eq(bluetoothSettings.userId, userId))
      .limit(1);

    if (settings.length === 0) {
      // Create default settings
      const defaultSettings = {
        userId,
        autoConnect: true,
        voiceCommandsEnabled: true,
        emergencyMode: true,
        audioQuality: 'high' as const,
        connectionTimeout: 10000,
        retryAttempts: 3,
        trustedDevicesOnly: false,
        emergencyProtocols: {
          activationPhrase: "emergency help",
          actions: ["start_recording", "contact_attorney", "notify_family"],
          priority: "critical",
          autoRecord: true,
          contactEmergencyServices: false,
          notifyEmergencyContacts: true
        } as EmergencyProtocol,
        createdAt: new Date(),
        updatedAt: new Date()
      };

      await db.insert(bluetoothSettings).values(defaultSettings);
      return defaultSettings;
    }

    return settings[0];
  } catch (error) {
    console.error('Error fetching Bluetooth settings:', error);
    return null;
  }
}

/
 * Test audio connection quality
```

```
   */
  static async testAudioConnection(userId: string, deviceId: string): Promise<{
    success: boolean;
    quality: 'excellent' | 'good' | 'fair' | 'poor';
    latency: number;
    bitrate: number;
    recommendations: string[];
  }> {
    try {
      const device = await db
        .select()
        .from(bluetoothDevices)
        .where(and(
          eq(bluetoothDevices.id, deviceId),
          eq(bluetoothDevices.userId, userId)
        ))
        .limit(1);

      if (!device.length) {
        return {
          success: false,
          quality: 'poor',
          latency: 0,
          bitrate: 0,
          recommendations: ['Device not found']
        };
      }

      const bluetoothDevice = device[0];
                                                const        quality        =
this.determineAudioQuality(bluetoothDevice.signalStrength);
      const latency = this.calculateLatency(quality);
      const bitrate = this.calculateBitrate(quality);

      const recommendations = [];
      if (quality === 'poor' || quality === 'fair') {
        recommendations.push('Move closer to device');
        recommendations.push('Check for interference');
        recommendations.push('Restart Bluetooth connection');
      }
      if (latency > 100) {
           recommendations.push('High latency detected - consider using wired
connection for critical audio');
      }

      return {
        success: true,
```

```
        quality,
        latency,
        bitrate,
        recommendations
      };
    } catch (error) {
      console.error('Error testing audio connection:', error);
      return {
        success: false,
        quality: 'poor',
        latency: 0,
        bitrate: 0,
        recommendations: ['Connection test failed']
      };
    }
  }

  private static calculateLatency(quality: string): number {
    switch (quality) {
      case 'excellent': return Math.random() * 20 + 10; // 10-30ms
      case 'good': return Math.random() * 30 + 30; // 30-60ms
      case 'fair': return Math.random() * 50 + 60; // 60-110ms
      case 'poor': return Math.random() * 100 + 100; // 100-200ms
      default: return 150;
    }
  }

  private static calculateBitrate(quality: string): number {
    switch (quality) {
      case 'excellent': return Math.random() * 64 + 256; // 256-320 kbps
      case 'good': return Math.random() * 64 + 192; // 192-256 kbps
      case 'fair': return Math.random() * 64 + 128; // 128-192 kbps
      case 'poor': return Math.random() * 64 + 64; // 64-128 kbps
      default: return 128;
    }
  }
}
```

MIDDLE SECTION SAMPLES - DATABASE SCHEMA & API ARCHITECTURE

Page 11-15: Comprehensive Database Schema (shared/schema.ts)

```typescript
import {
  pgTable,
  text,
  varchar,
  timestamp,
  jsonb,
  index,
  boolean,
  integer,
  uuid,
  decimal,
  primaryKey
} from "drizzle-orm/pg-core";
import { createInsertSchema, createSelectSchema } from "drizzle-zod";
import { z } from "zod";

// Session storage table for Replit Auth
export const sessions = pgTable(
  "sessions",
  {
    sid: varchar("sid").primaryKey(),
    sess: jsonb("sess").notNull(),
    expire: timestamp("expire").notNull(),
  },
  (table) => [index("IDX_session_expire").on(table.expire)],
);

// User storage table for Replit Auth
export const users = pgTable("users", {
  id: varchar("id").primaryKey().notNull(),
  email: varchar("email").unique(),
  firstName: varchar("first_name"),
  lastName: varchar("last_name"),
  profileImageUrl: varchar("profile_image_url"),
  subscriptionTier: varchar("subscription_tier").default("community_guardian"),
  hasAcceptedTerms: boolean("has_accepted_terms").default(false),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// Legal Rights Database - Core feature of C.A.R.E.N.
export const legalRights = pgTable("legal_rights", {
  id: uuid("id").primaryKey().defaultRandom(),
  stateCode: varchar("state_code", { length: 2 }).notNull(),
   category: varchar("category").notNull(), // Traffic Stops, Recording Rights,
```

```
etc.
  title: varchar("title").notNull(),
  content: text("content").notNull(),
  severity: varchar("severity").notNull(), // High, Medium, Low
  examples: text("examples").array(),
  relatedRights: varchar("related_rights").array(),
  legalCitations: varchar("legal_citations").array(),
  caseLaw: varchar("case_law").array(),
  enforceability: varchar("enforceability").notNull(),
  lastUpdated: timestamp("last_updated").defaultNow(),
  isActive: boolean("is_active").default(true),
});

// GPS Location tracking for incidents
export const locationData = pgTable("location_data", {
  id: uuid("id").primaryKey().defaultRandom(),
  latitude: decimal("latitude", { precision: 10, scale: 8 }).notNull(),
  longitude: decimal("longitude", { precision: 11, scale: 8 }).notNull(),
  address: text("address"),
  city: varchar("city"),
  state: varchar("state"),
  zipCode: varchar("zip_code"),
  county: varchar("county"),
  accuracy: integer("accuracy"), // GPS accuracy in meters
  timestamp: timestamp("timestamp").defaultNow(),
});

// Incident Recording System
export const incidents = pgTable("incidents", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: varchar("user_id").notNull(),
  type: varchar("type").notNull(), // traffic_stop, police_encounter, etc.
  status: varchar("status").default("active"), // active, completed, archived
  locationId: uuid("location_id").references(() => locationData.id),
  startTime: timestamp("start_time").defaultNow(),
  endTime: timestamp("end_time"),
  duration: integer("duration"), // in seconds
  description: text("description"),
  officerBadgeNumber: varchar("officer_badge_number"),
  officerName: varchar("officer_name"),
  departmentName: varchar("department_name"),
  vehicleInfo: jsonb("vehicle_info"), // license plate, make, model, etc.
  weatherConditions: varchar("weather_conditions"),
  roadConditions: varchar("road_conditions"),
  witnessInfo: jsonb("witness_info"),
  audioRecordingUrl: text("audio_recording_url"),
```

```
  videoRecordingUrl: text("video_recording_url"),
  evidenceFiles: text("evidence_files").array(),
  legalRightsInvoked: varchar("legal_rights_invoked").array(),
                                                        constitutionalRightsInvoked:
varchar("constitutional_rights_invoked").array(),
  searchConducted: boolean("search_conducted").default(false),
  searchConsented: boolean("search_consented").default(false),
  arrestMade: boolean("arrest_made").default(false),
  citationIssued: boolean("citation_issued").default(false),
  useOfForce: boolean("use_of_force").default(false),
                                                          emergencyContactsNotified:
boolean("emergency_contacts_notified").default(false),
  attorneyContacted: boolean("attorney_contacted").default(false),
  metadata: jsonb("metadata"), // Additional structured data
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// Attorney Network System
export const attorneys = pgTable("attorneys", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: varchar("user_id"), // If attorney has user account
  firmName: varchar("firm_name").notNull(),
  contactInfo: jsonb("contact_info").notNull(), // name, phone, email, address
  specializations: varchar("specializations").array(),
  barNumber: varchar("bar_number"),
  statesLicensed: varchar("states_licensed").array(),
  yearsExperience: integer("years_experience"),
  rating: decimal("rating", { precision: 3, scale: 2 }),
  availableForEmergency: boolean("available_for_emergency").default(false),
  hourlyRate: decimal("hourly_rate", { precision: 8, scale: 2 }),
  emergencyRate: decimal("emergency_rate", { precision: 8, scale: 2 }),
  responseTime: integer("response_time"), // average response time in minutes
  languages: varchar("languages").array(),
  profileImageUrl: text("profile_image_url"),
  bio: text("bio"),
  credentials: jsonb("credentials"),
  isVerified: boolean("is_verified").default(false),
  isActive: boolean("is_active").default(true),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// Emergency Contacts System
export const emergencyContacts = pgTable("emergency_contacts", {
  id: uuid("id").primaryKey().defaultRandom(),
```

```
  userId: varchar("user_id").notNull(),
  name: varchar("name").notNull(),
  relationship: varchar("relationship").notNull(),
  phoneNumber: varchar("phone_number").notNull(),
  email: varchar("email"),
  priority: integer("priority").default(1), // 1 = highest priority
   notificationPreferences:  jsonb("notification_preferences"),  //  sms,  email,
call
  isActive: boolean("is_active").default(true),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// Family Protection Coordination
export const familyMembers = pgTable("family_members", {
  id: uuid("id").primaryKey().defaultRandom(),
  primaryUserId: varchar("primary_user_id").notNull(),
  memberUserId: varchar("member_user_id").notNull(),
  relationship: varchar("relationship").notNull(),
  permissions: jsonb("permissions"), // what they can access/do
  canReceiveAlerts: boolean("can_receive_alerts").default(true),
  canViewIncidents: boolean("can_view_incidents").default(false),
  canContactAttorneys: boolean("can_contact_attorneys").default(false),
  isActive: boolean("is_active").default(true),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// Bluetooth Device Management
export const bluetoothDevices = pgTable("bluetooth_devices", {
  id: varchar("id").primaryKey(),
  userId: varchar("user_id").notNull(),
  deviceName: varchar("device_name").notNull(),
     deviceType:  varchar("device_type").notNull(),  //  earpiece,  headphones,
car_audio
  deviceAddress: varchar("device_address").notNull(),
  isConnected: boolean("is_connected").default(false),
  batteryLevel: integer("battery_level"),
  signalStrength: integer("signal_strength").default(0),
  capabilities: jsonb("capabilities"), // audio, microphone, voice commands
      trustLevel:   varchar("trust_level").default("new"),   //   trusted,   new,
suspicious
  lastConnected: timestamp("last_connected"),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});
```

```
// Voice Command History
export const voiceCommands = pgTable("voice_commands", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: varchar("user_id").notNull(),
  command: text("command").notNull(),
  action: varchar("action").notNull(),
  confidence: decimal("confidence", { precision: 4, scale: 3 }),
  context: jsonb("context"), // location, time, incident_id if applicable
  successful: boolean("successful").default(true),
  errorMessage: text("error_message"),
  executionTime: integer("execution_time"), // milliseconds
  deviceId: varchar("device_id"), // which device captured the command
  timestamp: timestamp("timestamp").defaultNow(),
});

// AI Learning System for continuous improvement
export const aiLearningData = pgTable("ai_learning_data", {
  id: uuid("id").primaryKey().defaultRandom(),
  incidentId: uuid("incident_id").references(() => incidents.id),
  patternType: varchar("pattern_type").notNull(),
  patternData: jsonb("pattern_data").notNull(),
  frequency: integer("frequency").default(1),
  stateCode: varchar("state_code"),
  legalCategory: varchar("legal_category"),
  confidenceScore: decimal("confidence_score", { precision: 4, scale: 3 }),
  recommendedAction: text("recommended_action"),
  isApproved: boolean("is_approved").default(false),
  appliedToDatabase: boolean("applied_to_database").default(false),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// Subscription Management
export const subscriptions = pgTable("subscriptions", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: varchar("user_id").notNull(),
  tier: varchar("tier").notNull(), // community_guardian, legal_shield, etc.
  status: varchar("status").notNull(), // active, cancelled, expired
  stripeCustomerId: varchar("stripe_customer_id"),
  stripeSubscriptionId: varchar("stripe_subscription_id"),
  currentPeriodStart: timestamp("current_period_start"),
  currentPeriodEnd: timestamp("current_period_end"),
  cancelAtPeriodEnd: boolean("cancel_at_period_end").default(false),
  features: jsonb("features"), // what features are included
  usageLimits: jsonb("usage_limits"), // device limits, recording limits, etc.
  createdAt: timestamp("created_at").defaultNow(),
```

```
  updatedAt: timestamp("updated_at").defaultNow(),
});

// Type exports for TypeScript
export type User = typeof users.$inferSelect;
export type UpsertUser = typeof users.$inferInsert;
export type LegalRights = typeof legalRights.$inferSelect;
export type InsertLegalRights = typeof legalRights.$inferInsert;
export type Incident = typeof incidents.$inferSelect;
export type InsertIncident = typeof incidents.$inferInsert;
export type Attorney = typeof attorneys.$inferSelect;
export type InsertAttorney = typeof attorneys.$inferInsert;
export type EmergencyContact = typeof emergencyContacts.$inferSelect;
export type InsertEmergencyContact = typeof emergencyContacts.$inferInsert;
export type BluetoothDevice = typeof bluetoothDevices.$inferSelect;
export type InsertBluetoothDevice = typeof bluetoothDevices.$inferInsert;
export type VoiceCommand = typeof voiceCommands.$inferSelect;
export type InsertVoiceCommand = typeof voiceCommands.$inferInsert;

// Zod schemas for validation
export const insertUserSchema = createInsertSchema(users);
export const selectUserSchema = createSelectSchema(users);
export const insertIncidentSchema = createInsertSchema(incidents);
export const selectIncidentSchema = createSelectSchema(incidents);
export const insertLegalRightsSchema = createInsertSchema(legalRights);
export const selectLegalRightsSchema = createSelectSchema(legalRights);
```

LAST 25 PAGES - USER INTERFACE & FRONTEND IMPLEMENTATION

Page 20-25: Main Application Router & UI Components (client/src/App.tsx)

```typescript
import { Switch, Route } from 'wouter';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { useAuth } from '@/hooks/useAuth';
import { Toaster } from '@/components/ui/toaster';
import { ThemeProvider } from '@/components/theme-provider';

// Core Pages
import Landing from '@/pages/Landing';
import Dashboard from '@/pages/Dashboard';
import SignIn from '@/pages/SignIn';
import DemoLogin from '@/pages/DemoLogin';

// Legal Protection Features
import LegalRights from '@/pages/LegalRights';
import InteractiveLegalMap from '@/pages/InteractiveLegalMap';
import AttorneyMessages from '@/pages/AttorneyMessages';
import AttorneyContact from '@/pages/AttorneyContact';

// Emergency Features
import EmergencyPullover from '@/pages/EmergencyPullover';
import Record from '@/pages/Record';
import EmergencyContactsManager from '@/pages/EmergencyContactsManager';

// Device & Settings
import DeviceSetup from '@/pages/DeviceSetup';
import BluetoothManager from '@/pages/BluetoothManager';
import Settings from '@/pages/Settings';
import AccountSecurity from '@/pages/AccountSecurity';

// Advanced Features
import EvidenceManager from '@/pages/EvidenceManager';
import VehicleReadability from '@/pages/VehicleReadability';
import MobilePerformance from '@/pages/MobilePerformance';
import AccessibilityEnhancer from '@/pages/AccessibilityEnhancer';
import LivestreamAttorneys from '@/pages/LivestreamAttorneys';

// AI & Learning Features
import AILearningDashboard from '@/pages/AILearningDashboard';

// Help & Support
import Help from '@/pages/Help';
import NotFound from '@/pages/NotFound';

// Layout Components
import SimplifiedSidebar from '@/components/SimplifiedSidebar';

// Create query client
const queryClient = new QueryClient({
```

```
  defaultOptions: {
    queries: {
      staleTime: 1000  60  5, // 5 minutes
      retry: 1,
    },
  },
});

function Router() {
  const { isAuthenticated, isLoading } = useAuth();

  // Loading state
  if (isLoading) {
    return (


Loading C.A.R.E.N...


    );
  }

  // Unauthenticated routes
  if (!isAuthenticated) {
    return (


    );
  }

  // Authenticated routes with sidebar layout
  return (
```

```
            {/ Dashboard /}

            {/ Emergency Features /}



            {/ Legal Protection /}



            {/ Device Setup /}



            {/ Advanced Features /}



            {/ Settings /}



            {/ 404 /}



    );
}
```

```
export default function App() {
  return (









  );
}
```

This source code package demonstrates the unique copyrightable expression in C.A.R.E.N.'s implementation, including proprietary algorithms for GPS-legal correlation, voice command recognition, multi-device coordination, and comprehensive database architecture for legal protection services.